

# LIBRARY MANAGEMENT AND BOOK CIRCULATION TRACKING SYSTEM

Java Console-Based Application

## PROJECT REPORT

|                      |                       |
|----------------------|-----------------------|
| <b>Student Name</b>  | DEVANSH SINGH         |
| <b>Roll Number</b>   | 25BAI10599            |
| <b>Course</b>        | Programming in Java   |
| <b>Year</b>          | Second Year           |
| <b>Institution</b>   | VIT Bhopal University |
| <b>Academic Year</b> | 2026–2027             |

Prepared according to the VITyarthi Build Your Own Project submission requirements.

# 1. Introduction

TheLibraryManagement and Book Circulation Tracking System is a Java-based console application for managing books, students, issue and return transactions, overdue periods, fines, and borrowing history. The application demonstrates core Java programming concepts through a modular and maintainable structure.

CSV files are used for local persistent storage so that records remain available after the application is closed.

## 2. Problem Statement

Manual library records can make it difficult to track book availability, current borrowers, due dates, overdue periods, fines, and complete borrowing history. The proposed system automates these operations and maintains consistent records.

## 3. Objectives

- Maintain books using unique Book IDs.
- Maintain students using unique Student IDs.
- Issue and return books through a controlled workflow.
- Automatically calculate a 14-day free period.
- Automatically calculate ₹10 per overdue day.
- Search a book and identify its current borrower when issued.
- Maintain historical issue and return records.
- Generate overdue, fine and most-borrowed-book reports.

## 4. Scope

The system covers book management, student management, book search, issue/return, automatic fine calculation, borrowing history, reporting, validation and CSV persistence. It is designed as a second-year Java project. Online payments, mobile applications, cloud deployment and enterprise authentication are outside the current scope.

## 5. Functional Requirements

| ID  | Module             | Requirement                                                |
|-----|--------------------|------------------------------------------------------------|
| FR1 | Book Management    | Add books and view books with availability.                |
| FR2 | Book Search        | Search by Book ID and display circulation details.         |
| FR3 | Student Management | Register and view students.                                |
| FR4 | Issue              | Issue an available book to a registered student.           |
| FR5 | Return             | Return a book and calculate the applicable fine.           |
| FR6 | Fine               | 14 free days; ₹10 for each overdue day.                    |
| FR7 | History            | Store and display borrowing history.                       |
| FR8 | Reports            | Show issued/overdue counts, fines and most-borrowed books. |

## 6. Non-Functional Requirements

- Usability: simple menu-driven interaction.
- Reliability: invalid operations are rejected.
- Error Handling: invalid input does not crash the application.
- Maintainability: separate model, service, storage and utility classes.
- Resource Efficiency: local CSV files and standard Java collections are used.
- Scalability: new books, students and records can be added.

## 7. System Architecture

The application uses a simple layered architecture: console interface → service/business logic → model objects and storage → CSV files.

|                                             |
|---------------------------------------------|
| USER / LIBRARIAN                            |
| Main.java — Console Interface               |
| LibraryService — Business Logic             |
| Book / Student / BorrowRecord — Model Layer |
| DataManager — Storage Layer                 |
| CSV Files — Persistent Data                 |

## 8. Workflow Diagram

|                       |
|-----------------------|
| START                 |
| Load CSV Data         |
| Display Main Menu     |
| Select Operation      |
| Validate Input        |
| Execute Operation     |
| Update Records        |
| Save CSV Data         |
| Return to Menu / Exit |

## 9. Use Case Diagram

Primary actor: Librarian / System User.

| Actor            | Use Cases                                                          |
|------------------|--------------------------------------------------------------------|
| Librarian / User | Add Book; View Books; Search Book; Register Student; View Students |
| Librarian / User | Issue Book; Return Book; Borrowing History; Reports                |

| Actor  | Use Cases                                                        |
|--------|------------------------------------------------------------------|
| System | Validate input; calculate due date; calculate fine; save records |

## 10. Class Diagram / Component View

| Class                     | Responsibility                                                                     |
|---------------------------|------------------------------------------------------------------------------------|
| Book                      | Book ID, title, author and availability.                                           |
| Student                   | Student ID and student name.                                                       |
| BorrowRecord              | Book/student relationship, issue date, due date, return date and fine calculation. |
| LibraryService            | Coordinates book, student, issue, return, history and reports.                     |
| DataManager               | Loads and saves CSV records.                                                       |
| InputValidator            | Validates menu values, IDs, text and dates.                                        |
| DateUtil / FineCalculator | Date display and 14-day/10 fine policy.                                            |

## 11. Sequence Diagram — Issue Book

| Step | Interaction                                         |
|------|-----------------------------------------------------|
| 1    | User → Main: Select Issue Book                      |
| 2    | Main → InputValidator: Read Book ID                 |
| 3    | Main → LibraryService: issueBook()                  |
| 4    | LibraryService → Book: Check availability           |
| 5    | LibraryService → Student: Check registration        |
| 6    | LibraryService → BorrowRecord: Create transaction   |
| 7    | LibraryService → DataManager: Save data             |
| 8    | LibraryService → Main: Display success and due date |

## 12. Storage / ER Design

The application stores data in three CSV files. The logical relationships are: Student 1—many BorrowRecords and Book 1—many BorrowRecords over time. An active BorrowRecord identifies the current borrower of an issued book.

| File               | Fields                                            |
|--------------------|---------------------------------------------------|
| books.csv          | bookId, title, author, available                  |
| students.csv       | studentId, name                                   |
| borrow_history.csv | bookId, studentId, issueDate, dueDate, returnDate |

## 13. Design Decisions & Rationale

- Console interface keeps the project focused on Java programming concepts.
- Packages separate responsibilities and improve maintainability.
- CSV persistence avoids external database-server configuration.
- LocalDate provides reliable date arithmetic.
- ArrayList provides simple in-memory record management.
- Automatic fine calculation demonstrates real-world business logic.

## 14. Implementation Details

### 14.1 Project Structure

```
src/library/ ■■■ Main.java ■■■ model/ ■■■■ Book.java ■■■■ Student.java ■■■■
BorrowRecord.java ■■■ service/ ■■■■ LibraryService.java ■■■ storage/ ■■■■
DataManager.java ■■■ util/ ■■■ InputValidator.java ■■■ DateUtil.java ■■■
FineCalculator.java
```

### 14.2 Fine Calculation

**Due Date = Issue Date + 14 days**

**Fine = Overdue Days × ■10**

If a book is returned on or before the due date, the fine is ■0. If returned later, only the overdue days are charged.

## 15. Testing Approach

| Test | Condition                 | Expected Result                                        |
|------|---------------------------|--------------------------------------------------------|
| T1   | Unique Book ID            | Book added and saved.                                  |
| T2   | Duplicate Book ID         | Operation rejected.                                    |
| T3   | Search existing Book ID   | Book details displayed.                                |
| T4   | Issue available book      | Transaction created; due date is issue date + 14 days. |
| T5   | Issue already-issued book | Operation rejected.                                    |
| T6   | Return on/before due date | Fine is ■0.                                            |
| T7   | Return after due date     | Fine = overdue days × ■10.                             |
| T8   | Restart application       | Saved CSV records are loaded.                          |
| T9   | Invalid menu/date input   | Error message shown without crash.                     |

## 16. Screenshots / Results

Insert screenshots from the actual VS Code execution before final submission. Recommended screenshots:

- Main menu after successful launch.
- Book list with available/issued status.
- Book search showing current borrower and due date.
- Issue transaction showing the 14-day due date.
- Return transaction showing overdue days and fine.
- Borrowing history.

- Reports showing overdue books and most-borrowed books.

## 17. Challenges Faced

- Keeping CSV data consistent with in-memory objects.
- Handling invalid input without program termination.
- Calculating dates and overdue periods correctly.
- Preventing duplicate IDs and invalid transactions.
- Maintaining modularity rather than placing all logic in Main.java.

## 18. Learnings & Key Takeaways

- Object-oriented programming and encapsulation.
- Package-based project organization.
- Java collections and file I/O.
- java.time date processing.
- Input validation and error handling.
- Separation of user interface, business logic and storage.
- Designing a project from functional and non-functional requirements.

## 19. Future Enhancements

- SQLite/MySQL integration using JDBC.
- Admin/librarian login.
- JavaFX or Swing GUI.
- Search by title/author/category.
- Book reservation and waitlist.
- Exportable reports.
- JUnit unit testing.
- GitHub version history and releases.

## 20. GitHub Repository Plan

The repository should contain the source packages, data/configuration files, README.md and statement.md. The README should document the title, overview, features, technologies, installation/run steps, testing instructions and screenshots.

```
LibraryManagementSystem/ ■■■ src/ ■■■ data/ ■■■ README.md ■■■ statement.md ■■■ .gitignore
```

## 21. References

1. VITYarhi — Build Your Own Project: General Project Instructions & Submission Guidelines (provided course document).
2. Java standard library APIs used by the project, including java.time, java.util and java.nio.file.
3. Project source code prepared for this submission.

## Appendix — Example Output

```
===== LIBRARY MANAGEMENT SYSTEM
===== 1. Add Book 2. View All Books 3. Search Book /
Check Status 4. Register Student 5. View All Students 6. Issue Book 7. Return Book 8.
Borrowing History 9. Reports & Fine Summary 10. Save & Exit
```

Example fine: due date 15-09-2026, return date 18-09-2026 → 3 overdue days → ■30 fine.